RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

PROJECT REPORT

MENU-DRIVEN CALCULATOR

Problem Statement No. 25

Language: Python 3

Student Name	Saikat Jana
ERP Number	00353
Course	BCA (Artificial Intelligence and Machine Learning)
Section	1-B
University	Rungta International Skills University

Table of Contents

1. Problem Definition

2. Objectives

3. Requirements

4. Concepts Used

5. Algorithm

6. Flowchart

7. Program Design (Functions)

8. Source Code

9. Test Cases and Output

10. Requirement Checklist

11. Conclusion

12. Future Scope

1. Problem Definition

Create a console calculator that repeatedly shows a menu of arithmetic operations, takes two numbers, displays the result and handles errors such as division by zero until the user chooses to exit.

The menu has seven options: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus and 7 Exit. All results are rounded to 2 decimal places.

2. Objectives

- Build a loop-driven menu that keeps running until the user selects Exit.
- Write a separate function for each operation to keep the code modular and reusable.
- Validate the menu choice and reject anything outside 1 to 7.
- Handle division by zero and non-numeric input with *try-except* so the program never crashes.
- Display every result rounded to 2 decimal places.

3. Requirements

Software Requirements

- Python 3.8 or higher
- Any terminal / command prompt (Windows, Linux or macOS)
- Any text editor or IDE (VS Code, PyCharm, IDLE)

Hardware Requirements

- Any computer capable of running Python 3 (no special hardware needed)

4. Concepts Used

Concept	Where it is used
Functions	add(), subtract(), multiply(), divide(), power(), modulus(), read_choice(), read_numbers(), main()
Loops	*while True* loop in main() shows the menu again and again until Exit
Operators	Arithmetic operators + - * / ** % and comparison operators == <= >=
Exceptions	try-except for ValueError, ZeroDivisionError and OverflowError
Dictionary	OPERATIONS maps each menu choice to its function and symbol
String formatting	f-strings with :.2f for 2 decimal places

5. Algorithm

Step 1: START.

Step 2: Display the menu (1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit).

Step 3: Read the user's choice. If it is not an integer between 1 and 7, print *Invalid choice. Select 1 to 7.* and go back to step 2.

Step 4: If the choice is 7, print *Thank you for using the calculator. Exiting...* and go to step 9.

Step 5: Read two numbers from the user.

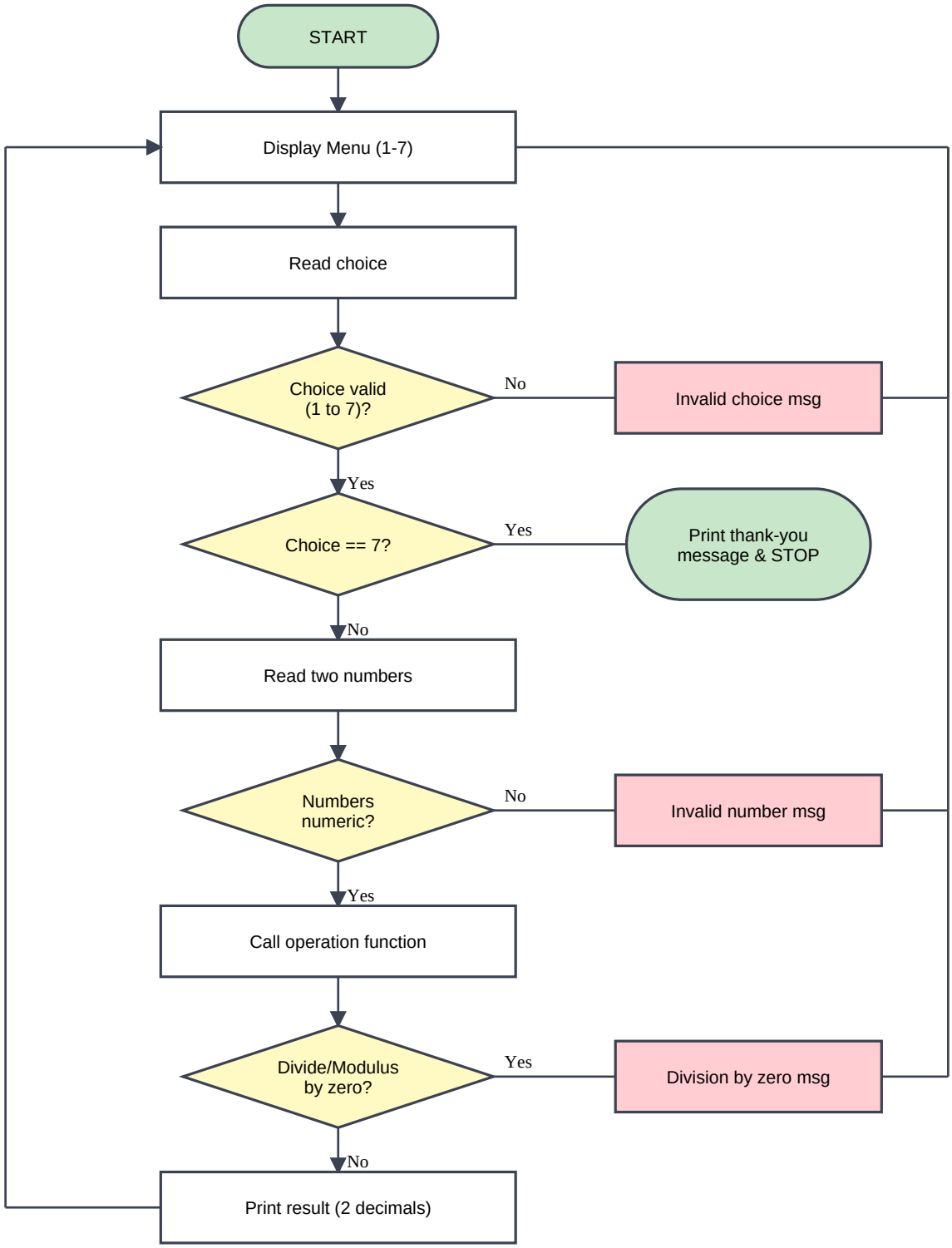
Step 6: If either value is not numeric, print *Invalid number. Enter numeric values only.* and go back to step 2.

Step 7: Call the function that matches the choice. If the operation is Divide or Modulus and the second number is 0, print *Error: Division by zero is not allowed.* and go back to step 2.

Step 8: Print the expression and the result rounded to 2 decimal places, then go back to step 2.

Step 9: STOP.

6. Flowchart



The program returns to the menu after every operation or error and ends only when the user selects option 7.

7. Program Design (Functions)

Function	Purpose
`add(a, b)`	Returns a + b
`subtract(a, b)`	Returns a - b
`multiply(a, b)`	Returns a * b
`divide(a, b)`	Returns a / b; raises ZeroDivisionError if b is 0
`power(a, b)`	Returns a raised to the power b (a ** b)
`modulus(a, b)`	Returns a % b; raises ZeroDivisionError if b is 0
`read_choice()`	Reads and validates the menu choice; returns 1-7 or None
`read_numbers()`	Reads two inputs and converts them to float; raises ValueError for non-numeric input
`fmt(n)`	Formats operands neatly (12.0 becomes 12)
`main()`	Runs the menu loop, calls the operation and prints results or errors

8. Source Code

File: `code.py`

```
"""
Project  : PS 25 - Menu-Driven Calculator
Author   : Saikat Jana
ERP No.  : 00353
Course   : BCA (Artificial Intelligence and Machine Learning)
Section  : 1-B
University: Rungta International Skills University

A console calculator that repeatedly shows a menu of arithmetic
operations, takes two numbers, displays the result (rounded to 2 decimal
places) and handles errors such as division by zero until the user exits.
"""

MENU = """
===== CALCULATOR MENU =====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Power
6. Modulus
7. Exit"""


# ---------- One function for each operation ----------
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Division by zero is not allowed.")
    return a / b


def power(a, b):
    return a ** b


def modulus(a, b):
    if b == 0:
        raise ZeroDivisionError("Division by zero is not allowed.")
    return a % b
```

```
# choice -> (function, symbol)
OPERATIONS = {
    1: (add, "+"),
    2: (subtract, "-"),
    3: (multiply, "*"),
    4: (divide, "/"),
    5: (power, "^"),
    6: (modulus, "%"),
}


def read_choice():
    """Read and validate the menu choice. Returns an int 1-7 or None."""
    raw = input("Enter your choice (1-7): ").strip()
    try:
        choice = int(raw)
    except ValueError:
        return None
    return choice if 1 <= choice <= 7 else None


def read_numbers():
    """Read two numbers. Raises ValueError if either is not numeric."""
    a = float(input("Enter first number: ").strip())
    b = float(input("Enter second number: ").strip())
    return a, b


def fmt(n):
    """Show operands cleanly: 12.0 -> 12, 9.5 -> 9.5"""
    return f"{n:g}"


def main():
    while True:
        print(MENU)
        choice = read_choice()

        if choice is None:
            print("Invalid choice. Select 1 to 7.")
            continue

        if choice == 7:
            print("Thank you for using the calculator. Exiting...")
            break

        func, symbol = OPERATIONS[choice]

        try:
            a, b = read_numbers()
            result = func(a, b)
```

```
            if isinstance(result, complex):
                print("Error: Result is not a real number.")
                continue
            print(f"{fmt(a)} {symbol} {fmt(b)} = {result:.2f}")
        except ZeroDivisionError as err:
            print(f"Error: {err}")
        except ValueError:
            print("Invalid number. Enter numeric values only.")
        except OverflowError:
            print("Error: Result is too large to calculate.")


if __name__ == "__main__":
    main()
```

9. Test Cases and Output

All four test cases from the problem statement were executed using the program. The terminal output shown below is the real output captured from running `code.py`. To keep it short, the menu block is shown as a single line.

Test Case 1: Normal Input

INPUT	EXPECTED OUTPUT
Option 1 (Add): 12 and 8	12 + 8 = 20.00

Actual program output:

```
[ MENU shown: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit ]
Enter your choice (1-7): 1
Enter first number: 12
Enter second number: 8
12 + 8 = 20.00

[ MENU shown again ]
Enter your choice (1-7): 7
Thank you for using the calculator. Exiting...
```

Result: PASS – actual output matches the expected output.

Test Case 2: Boundary / Minimum Input

INPUT	EXPECTED OUTPUT
Option 4 (Divide): 0 and 5 (zero numerator)	0 / 5 = 0.00

Actual program output:

```
[ MENU shown: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit ]
Enter your choice (1-7): 4
Enter first number: 0
Enter second number: 5
0 / 5 = 0.00

[ MENU shown again ]
Enter your choice (1-7): 7
Thank you for using the calculator. Exiting...
```

Result: PASS – actual output matches the expected output.

Test Case 3: Invalid Input

INPUT	EXPECTED OUTPUT
(a) Option 4 (Divide): 10 and 0 (b) Option 3 (Multiply): abc and 4 (c) Menu choice: 9	(a) Error: Division by zero is not allowed. (b) Invalid number. Enter numeric values only. (c) Invalid choice. Select 1 to 7.

Actual program output:

```
[ MENU shown: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit ]
Enter your choice (1-7): 4
Enter first number: 10
Enter second number: 0
Error: Division by zero is not allowed.

[ MENU shown again ]
Enter your choice (1-7): 3
Enter first number: abc
Invalid number. Enter numeric values only.

[ MENU shown again ]
Enter your choice (1-7): 9
Invalid choice. Select 1 to 7.

[ MENU shown again ]
Enter your choice (1-7): 7
Thank you for using the calculator. Exiting...
```

Result: PASS – actual output matches the expected output.

Test Case 4: Different Valid Scenario

INPUT	EXPECTED OUTPUT
(a) Option 5 (Power): 2 and 10 (b) Option 6 (Modulus): 17 and 5 (c) Option 2 (Subtract): 5 and 9.5 (d) Option 7 (Exit)	(a) 2 ^ 10 = 1024.00 (b) 17 % 5 = 2.00 (c) 5 - 9.5 = -4.50 (d) Thank you for using the calculator. Exiting...

Actual program output:

```
[ MENU shown: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit ]
Enter your choice (1-7): 5
Enter first number: 2
Enter second number: 10
2 ^ 10 = 1024.00

[ MENU shown again ]
Enter your choice (1-7): 6
Enter first number: 17
Enter second number: 5
17 % 5 = 2.00

[ MENU shown again ]
Enter your choice (1-7): 2
Enter first number: 5
Enter second number: 9.5
5 - 9.5 = -4.50

[ MENU shown again ]
Enter your choice (1-7): 7
Thank you for using the calculator. Exiting...
```

Result: PASS – actual output matches the expected output.

Additional Edge Cases Tested

Input	Output
Option 6 (Modulus): 5 and 0	Error: Division by zero is not allowed.
Option 5 (Power): -8 and 0.5	Error: Result is not a real number.
Option 5 (Power): 10 and 1000	Error: Result is too large to calculate.
Menu choice: q (text)	Invalid choice. Select 1 to 7.

10. Requirement Checklist

Bare Minimum Requirement	Status
Solution written in Python 3 (one .py file) and runs from the console	**Done**
Loop-driven menu with an Exit option	**Done**
One function for each operation (add, subtract, multiply, divide, power, modulus)	**Done**
Division by zero and non-numeric input handled using try-except	**Done**
Menu choice validated; every result rounded to 2 decimal places	**Done**
All 4 test cases pass	**Done**

11. Conclusion

The Menu-Driven Calculator project meets all the requirements given in the problem statement. It uses functions for each arithmetic operation, a loop to keep the menu running, operators for the calculations and exception handling to deal with division by zero and invalid input. The program does not crash on wrong input; it displays a clear message and returns to the menu. All four test cases produced exactly the expected output.

Through this project I practised dividing a problem into small functions, validating user input, using try-except and formatting output in Python.

12. Future Scope

- Add more operations such as square root, factorial, percentage and floor division.
- Store a history of calculations and save it to a text file.
- Add a graphical interface using Tkinter.
- Accept an expression such as 12 + 8 * 3 directly and evaluate it safely.